Pearls AQI Predictor

A Serverless Machine Learning System for 3-Day Air Quality Forecasting

Project Development Report

City of Focus: Karachi, Pakistan

Made by: Maham Imran

1. Introduction and Objectives

This report walks through how the Pearls AQI Predictor was built: the design decisions, the things that went wrong along the way, and how each one was fixed. The end result forecasts Karachi's Air Quality Index three days ahead, built as a fully serverless pipeline, independent, stateless jobs handle data collection, feature engineering, training, and serving, all coordinated by scheduling rather than any one server that has to stay running.

The goal from the start was to build something real, not a demo that only looks finished. That meant genuine data ingestion from live external sources, real feature engineering, an actual historical backfill, models that are trained and evaluated rather than assumed, a working feature store and model registry, automated retraining on a schedule, an explainability layer that computes real values, hazard alerting, and a dashboard where every number on screen can be traced back to an actual computation or API response, never a placeholder standing in for one.

2. Technology Stack

- Backend language: Python
- Machine learning: scikit-learn (Random Forest, Ridge Regression) and a XGBoost for a gradient boosted tree model
- Feature store and model registry: Hopsworks, with an automatic local fallback when cloud credentials are not configured
- Automation: scheduled workflows for hourly ingestion, daily retraining, and on-demand historical backfill
- Web API: a lightweight Python web framework exposing a REST interface
- External data sources: a ground-station air-quality network, a commercial weather API, and a free, keyless weather and air-quality archive
- Explainability: SHAP (Shapley Additive explanations)
- Version control: Git, hosted on GitHub, with automation triggered directly from the repository
- Frontend: A TypeScript single page application built with a modern component framework and a utility first styling system

3. System Architecture

The system is split into two independent parts that communicate only over HTTP. The backend owns all real data access, feature computation, model training, and storage; it exposes a small set of REST endpoints. The frontend holds no business logic of its own, it only renders whatever the backend returns. This separation means either half can be redeployed, restarted, or rebuilt without touching the other, and it made it possible to rebuild the frontend from scratch later in the project without any backend changes.

3.1 Data flow

- External APIs supply live and historical pollutant and weather readings
- A feature-engineering step computes time-based features (hour, day, month, weekday) and derived features (lagged AQI values, rolling statistics, rate of change)
- Engineered features are written to the feature store, which serves both training and live inference
- A training job reads the full historical feature set, trains three candidate model types per forecast horizon, evaluates them, and stores the best performing model per horizon
- The web API loads the current best models and the latest features to compute the three-day forecast on request

3.2 Hopsworks: Feature Store and Model Registry

Hopsworks was chosen as the managed feature store and model registry for this project. It serves two distinct roles here: A Feature Store, which holds the full engineered dataset (both the historical backfill and every hourly reading added since), and a Model Registry, used to persist trained model artefacts alongside their metrics.

The integration was built with a deliberate local fallback: whenever Hopsworks credentials are not configured, the same code path transparently writes to and reads from a local file instead, using an identical interface. This meant the rest of the system: training, prediction, the web API never needed to know or care which backend was active, and it made it possible to develop and test the full pipeline offline before any cloud account was involved.

In practice, the Hopsworks integration was also the single largest source of debugging effort in the project (see Section 6). Several of those issues were specific to the schema-versioning model Hopsworks uses for a feature group: once a version's schema and storage location are established, they are effectively fixed, so structural changes (such as adding weather columns that were not present in the original backfill) require moving to a new version number rather than altering the existing one in place. Over the course of development, the project moved through several version numbers as different storage-initialization issues were diagnosed and worked around, before settling on a stable, verified version that is now the system's single source of truth for feature data.

One practical limitation of this design is worth noting for anyone extending the project: the local dashboard's Model Registry view reads the most recent training results from a local file that is only refreshed when training runs on that same machine. When training instead runs on the scheduled automation, its results are written to Hopsworks (and to a downloadable run artefact) but do not automatically appear on a different machine's local dashboard until either training is re-run locally or the artefact is fetched manually. This is expanded on in the Limitations section.

3.3 Data sources

Three external providers feed the system, and which one is treated as authoritative depends on the situation. This split was not the original design, it came out of a data quality problem discovered partway through the project and is explained fully in Section 7.

Use case	Primary source	Secondary / fallback	Why
Live current reading (dashboard)	Open-Meteo (Air Quality + Forecast API)	AQICN, with OpenWeather for weather if AQICN is used	Open-Meteo reports real values across all six pollutant categories; the Karachi AQICN station only reports PM2.5, so relying on it alone left the other pollutants showing as zero
Hourly feature ingestion (training data)	Open-Meteo (Air Quality + Forecast API)	AQICN, used only as a staleness cross-check	AQICN updates on its own multi-hour cycle, so a stuck reading could otherwise be logged as several identical "fresh" hourly rows
Historical backfill	Open-Meteo (Air Quality Archive + Historical Weather Archive)	None currently required	The only source used that provides both historical pollutant data and historical weather data together, with no API key needed

4. Development Process

4.1 Backend pipeline

The backend was built first, as a standalone, independently testable system, before any frontend work began. It was structured as a set of small, single purpose modules:

- An API client module responsible for every external network call, isolating all third party integration in one place
- A feature engineering module computing time and derived features from raw readings
- A feature store module abstracting over the managed feature store and a local fallback, so the rest of the codebase never needs to know which one is active
- A backfill module reconstructing months of historical training data in one run
- An hourly feature ingestion module appending one new live reading per run
- A training module that fits, evaluates, and compares three model types per forecast horizon
- A model-registry module that stores trained models and tracks which candidate is currently active per horizon
- A prediction module producing the three-day forecast from the active models
- An explainability module computing SHAP feature importance for the active models
- An alerting module checking forecasts against a hazard threshold and notifying by email or chat webhook
- A REST API layer exposing all of the above to a frontend

4.2 Automated testing before deployment

Before relying on any live cloud service, the full pipeline: backfill, feature engineering, training, prediction, explainability, and alerting, was exercised end to end using synthetic but realistic data with the network layer substituted, so that every stage of the logic could be verified for correctness independently of external service availability. This caught several integration issues early, before they could appear in a live run.

4.3 Automation

Three scheduled workflows were configured: an hourly job that ingests one new live reading, a daily job that retrains all models, and a manually triggered job that performs a full historical backfill. All three run independently of any local machine.

4.4 Frontend development

The project initially had a separate, pre-existing frontend prototype that displayed entirely simulated values,AQI readings, model metrics, and explainability scores were all generated locally in the browser rather than computed from real data. This was identified early and treated as a defect: it would not have satisfied the requirement that every displayed number be genuine.

The prototype was removed entirely and a new frontend was built from scratch against the backend's actual REST contract: a current conditions view, a three-day forecast with both chart and table presentations, a feature-store view, a model-registry view with genuine model switching, an explainability view, a historical trends view, and an alerting panel. Every request in the new frontend maps to a documented backend endpoint, and the interface was scoped to a single city and to only the features explicitly required, removing pages that were not part of the requirements.

5. Model Training and Registry Mechanics

This section describes, in detail, which models the system actually trains, how they are evaluated and compared, and exactly how a trained model makes its way into the Hopsworks Model Registry versus staying local.

5.1 What gets trained

The system does not train one model. For each of the three forecast horizons (24 hours, 48 hours, and 72 hours ahead), three different candidate model types are trained independently and compared against each other. That means a single full training run produces nine trained models in total: three candidate types, each fit separately for each of the three horizons.

Candidate	Category	Library	Notes
Random Forest Regressor	Ensemble tree-based (statistical machine learning)	scikit-learn	300 trees, capped depth; generally the strongest performer on this dataset and the most common winner per horizon
Ridge Regression	Regularized linear regression (statistical machine learning)	scikit-learn	Fastest to train and explain; features are standardized before fitting
Recurrent neural network (LSTM)	Deep learning	TensorFlow / Keras	Two stacked LSTM layers with early stopping; the slowest to train and the slowest to explain, since neural networks require a sampling-based explanation method rather than an exact one
XGBoost	Gradient boosted trees, statistical machine learning	XGBoost	Replaced an earlier recurrent neural network candidate. Handles the tabular, feature engineered structure of this dataset well, trains quickly, and its predictions can be explained almost instantly

All three candidates for a given horizon are trained on an identical train/test split of the same historical feature set, so their scores are directly comparable. No candidate is given an advantage in terms of data access, the only thing that varies between them is the model architecture itself.

5.2 Evaluation and selection

Each candidate is scored on a held out test portion of the data using three metrics: RMSE (root mean squared error), MAE (mean absolute error), and R^2 (coefficient of determination). For each horizon, the candidate with the lowest RMSE on the test set is selected as the active model for that horizon this is the model the prediction and explainability endpoints will actually use until a future training run produces a better one, or until it is manually switched from the dashboard.

In practice, the Random Forest candidate has most often been the winner across horizons on this dataset, though the comparison is re-run from scratch on every training cycle, so this is not hard-coded a future run with more data or different conditions could just as easily select Ridge Regression or the recurrent network instead.

5.3 From a trained model to the Hopsworks Model Registry

Getting a model from "just finished training in memory" to "available in Hopsworks" happens in the sequence below. The key detail, easy to miss, is in step 3 and step 5: every candidate is saved locally, but only the single winning candidate per horizon is ever uploaded to Hopsworks.

Step	What happens	Where it lands
1. Train	All three candidates are trained independently for a given horizon, on the same train/test split of the full historical feature set	In memory, during the training run
2. Evaluate	Each candidate is scored on the held-out test portion using RMSE, MAE, and R^2	Metrics recorded per candidate
3. Save every candidate locally	All three trained candidates not just the best one are written to local files (a serialized model file plus a metrics file each), regardless of which will end up active	Local disk, one folder per candidate per horizon
4. Select the winner	The candidate with the lowest RMSE for that horizon is marked active	A local pointer file records which candidate is currently active per horizon
5. Push only the winner to Hopsworks	Only the winning candidate's model artefact and metrics are uploaded to the Hopsworks Model Registry	Hopsworks Model Registry (cloud)
6. Serve predictions	The prediction and explainability endpoints load whichever candidate the local pointer file currently marks as active for each horizon	Read from local disk at request time

5.4 Local storage layout

Locally, each candidate is stored in its own folder named after the candidate and horizon (for example, a folder for the Random Forest candidate at the 24-hour horizon, a separate one for the Ridge candidate at the same horizon, and so on), each containing the serialized model file and a small metrics file. A separate pointer file per horizon records only the name of whichever candidate is presently active; switching the active model from the dashboard rewrites that pointer rather than moving or retraining anything.

5.5 Practical implications

- **Only winners reach Hopsworks:** The two non-winning candidates for each horizon exist only on whichever machine trained them; they are never pushed to the cloud registry. This was a deliberate choice to avoid uploading models that will not be used, but it does mean the cloud registry is not a complete history of every model ever trained only of every model that was, at some point, the best for its horizon.
- **Switching the active model is a local operation:** The dashboard's "Set as Active" control changes which locally saved candidate is used to serve predictions on that machine. It does not push anything new to Hopsworks; the cloud registry continues to reflect whichever candidate most recently won a training run, regardless of what is active locally.
- **Retraining always re-evaluates all three candidates:** There is no persistent bias toward a previous winner every scheduled or manually triggered training run starts from scratch and lets the metrics decide again, on the training data available at that time.

5.6 Improving model performance

The first complete training runs produced disappointing results. R squared values across horizons sat roughly between 0.08 and 0.49, well short of the target a mentor had suggested as a reasonable bar for this kind of forecasting problem, an R squared of at least 0.7, an RMSE roughly between 20 and 30, and an MAE roughly between 10 and 20. Two changes were made in response, one to how the models are trained, and one to which models are used at all.

First, none of the three candidates had ever had their settings tuned, they were all trained with reasonable but arbitrary fixed defaults. A proper hyperparameter search was added for every candidate, using a randomized search over a time aware cross validation scheme rather than a plain train and test split, so the settings chosen are the ones that actually generalize across different stretches of the training period rather than ones that happen to fit a single split well.

Second, the recurrent neural network candidate was replaced with XGBoost, a gradient boosted tree model. Neural networks of this kind generally need considerably more data and more careful tuning than a dataset of this size naturally provides, and they add meaningful training and explanation time without a clear accuracy benefit here. XGBoost is a better natural fit for structured, feature engineered tabular data like lag values, rolling statistics, and time based features, tends to perform at least as well as a neural network on this kind of problem with far less tuning effort, and its predictions can be explained by SHAP almost instantly, unlike a neural network which needs a much slower, sampling based explanation method.

Alongside these two changes, diagnostic logging was added directly to the training pipeline rather than treating low accuracy as a black box to guess at. Two figures are now logged for every horizon, what share of the rows survive the cleaning step that removes incomplete rows, since a large drop here usually points at a missing feature for some stretch of the dataset rather than a modelling problem, and how the target variable's average and spread compare between the training period and the held out test period, since a large difference there points at a seasonal shift the model simply has not seen enough of yet rather than a bug. Duplicate timestamps are also removed before training, since the feature group's version history could otherwise leave more than one row for the same hour in the underlying store.

At the time of writing this improvement work is still in progress, the tuned models and the new candidate had been put in place and the diagnostics were being used to track down exactly which of the two likely causes, data loss during cleaning or a seasonal mismatch between the training and test periods, was the larger contributor for this dataset, with further backfill and adjustment planned as needed to close the remaining gap to the target metrics.

6. Challenges Encountered and Solutions

This section is the honest account of what actually went wrong along the way, not a cleaned-up summary. Getting Hopsworks working reliably from a mixed Windows-and-Linux setup, in particular, took far longer than expected and involved a fair amount of trial and error. Each problem below was worked through the same way: read the error carefully, try to reproduce it on purpose, fix the actual cause rather than patch around the symptom, then confirm the fix with a real run before moving on. They are listed roughly in the order they came up.

Issue	Root Cause	Resolution
The existing frontend prototype looked complete but was not actually connected to any real data	It generated AQI values, model metrics, and explanations locally in the browser instead of calling a backend	Discarded the prototype entirely and rebuilt the interface from scratch against the backend's real REST endpoints, verified endpoint by endpoint

Backend deployment on Windows failed while connecting to Hopsworks	The Hopsworks client library hard-codes a Unix-style temporary directory path when downloading connection certificates, which does not exist by default on Windows	Created the missing directory manually as an interim fix, and moved primary write operations to the Linux-based automation runners where the path resolves correctly
A required Hopsworks storage dependency (Delta Lake support) failed to install on Windows	The package needed to compile a native extension using a Rust toolchain not available in the Windows build environment	Restricted that dependency to Linux-only installation using an environment marker in the requirements file, so it only installs where it is actually needed
A pipeline run reported success even though no data had reached Hopsworks	A defensive fallback silently wrote to a local file when the Hopsworks write failed, and reported the run as successful	Removed the silent fallback for this code path; the pipeline now fails loudly and visibly when a write does not reach Hopsworks, so failures are never hidden
Hopsworks rejected feature inserts with a schema-mismatch error	The Hopsworks feature-group schema expected 32-bit integer columns, while the default numeric type produced by the data-processing library is 64-bit	Added an explicit type-conversion step before every insert that downcasts integer columns to the expected width
A later Hopsworks schema-mismatch error appeared for weather-related columns	The historical backfill data source did not originally provide weather fields, so the Hopsworks feature-group schema was created without them, while the hourly pipeline later began sending them	Aligned both pipelines: the backfill process was switched to a data source that supplies both pollutant and weather history, and the hourly pipeline was updated to only send columns already present in the established Hopsworks schema
The hourly pipeline crashed when combining historical and live readings	Timestamps read back from Hopsworks were timezone-naive, while newly generated readings were timezone-aware, and the two could not be combined directly	Standardised all timestamp handling to a single, explicit UTC-aware format before any comparison or combination
A Hopsworks feature-group version repeatedly failed to accept new data, even after retries	The underlying storage location for that version appears to have never initialised correctly on the Hopsworks side after an earlier failed write attempt	Moved to a new, previously unused Hopsworks schema version, which resolved the issue immediately; verified with a full historical load before proceeding
Large historical loads occasionally failed on the very first write attempt to Hopsworks	A single very large write appeared to strain the Hopsworks storage backend, particularly immediately after a new schema version was created	Split large historical loads into smaller batches with independent retry logic per batch, and added a short pause after creating a new Hopsworks schema version before writing to it
Automation workflows failed with a missing-file error after the project was restructured into separate frontend and backend folders	The automation configuration assumed all commands ran from the repository root, but the backend code and its dependency list had moved into a subfolder	Set an explicit working directory for every automation step so commands run from the correct subfolder

A single failed historical-data request silently produced a permanent gap in the training dataset	One failed attempt at fetching a date range was treated as final, with no retry, and the affected date range was simply skipped without a clear summary	Added retry logic for each individual date range before giving up on it, and added a clear end-of-run summary listing any date ranges that could not be recovered
The Karachi ground-station reading repeated the same AQI value across several consecutive hourly runs	The station updates on its own multi-hour cycle rather than every hour, but the pipeline treated every reading as fresh	Made Open-Meteo the primary source for both live and historical readings, kept the ground station only as a fallback, and added a staleness check that flags repeated identical values (see Section 7)
The live dashboard showed several pollutants as 0.0 µg/m³	The ground station used only measures PM2.5; the missing values for the other categories were being defaulted to zero instead of being treated as unavailable	Switched the current-reading endpoint to Open-Meteo as well, since it reports genuine measurements for all six pollutant categories
A local push to GitHub was rejected	The remote branch had commits (from automated runs writing back results) that did not exist locally yet	Pulled the remote changes first, resolved the one file that conflicted, then pushed again a routine Git workflow issue rather than a defect in the project itself
Dashboard load intermittently produced several concurrent Hopsworks connection errors	Every dashboard tab calls its own API endpoint, and several of them load in parallel. Each one independently tried to log in to Hopsworks and open a fresh Arrow Flight connection at the same time, and the underlying client library is not designed to have several logins happening at once from the same process	Replaced the per request connection with a single shared connection created once when the application starts, guarded by a lock so that two nearly simultaneous requests cannot create it twice. This is expanded on later in this section
Even after the shared connection was in place, the dashboard still occasionally hit connection errors that mentioned the end of a TCP stream	The shared connection solved concurrent logins, but several endpoints could still trigger independent reads from Hopsworks at the same moment if the cache had just expired	Added a dedicated lock around the actual read operation, so only one request performs a real Hopsworks read at a time while the others wait briefly and reuse that result once it is ready, and began warming the cache immediately when the application starts rather than waiting for the first dashboard visit
Model accuracy on the held out test set was far below the target the mentor had suggested	The three original candidates were trained with fixed, untuned settings, and one of them was a recurrent neural network that needs more data and more careful tuning than a small tabular dataset like this one naturally provides	Introduced a proper hyperparameter search for every candidate, added diagnostic logging that reports how much data survives cleaning and how the training and test periods compare statistically, and replaced the neural network candidate with a gradient boosted tree model that is a better natural fit for this kind of structured, feature engineered data. This is expanded on in Section 5.6

Looking back, most of these fall into one of two buckets: things that were genuinely temporary (a flaky connection, a storage service that needed a moment to catch up) and things that were never going to fix themselves no matter how many times they were retried (a schema mismatch, a missing column). Once that distinction became clear, the retry logic was written to match keep trying the first kind, fail fast and loud on the second kind and runs got both more reliable and quicker to debug when they did fail.

Concurrent Dashboard Requests and Arrow Flight Stability

During testing, the dashboard occasionally experienced multiple simultaneous FlightUnavailableError ("End of TCP stream") exceptions when loading. Investigation showed that although the Hopsworks connection was already implemented as a process-wide singleton to prevent concurrent logins, the feature-store read path was still unprotected. When the dashboard loaded, several API endpoints (Current AQI, Forecast, Feature Store, SHAP Analytics, and Model Registry) were requested in parallel. If the 5-minute feature-store cache was empty or had just expired, every request independently executed a new Arrow Flight query against the same Hopsworks Feature Group. These concurrent reads increased the likelihood of Arrow Flight connection failures on the free-tier Hopsworks cluster.

To improve reliability, the feature-store read path was synchronized using a dedicated read lock. This ensures that only one thread performs a real Hopsworks read at a time, while all other concurrent requests wait briefly and reuse the cached result once it becomes available. In addition, the feature-store cache is pre-warmed immediately after establishing the Hopsworks connection during application startup. This allows the first dashboard page load to use an already populated cache instead of triggering multiple simultaneous reads.

These improvements significantly reduced unnecessary load on the Hopsworks Query Service while preserving the existing retry mechanism and snapshot fallback. After implementation, the dashboard consistently served cached feature-store data during temporary Arrow Flight outages instead of generating multiple simultaneous connection failures, resulting in a more stable and responsive application.

Key Improvements

- Added a dedicated lock around the Hopsworks feature-store read operation.
- Prevented multiple concurrent Arrow Flight queries from dashboard requests.
- Pre-loaded the feature-store cache during application startup.
- Continued using retry logic with exponential backoff for transient failures.
- Retained the local snapshot fallback to ensure uninterrupted dashboard operation during temporary Hopsworks outages.
- Improved overall dashboard reliability and reduced unnecessary load on the Hopsworks Query Service.

7. Data Quality Improvement

While reviewing the hourly pipeline's output, something stood out: the ground-station AQI reading for Karachi kept showing up as the exact same number, hour after hour. It turned out this was expected behavior on the station's end these ground stations refresh on their own multi-hour cycle, not every hour but the pipeline had no way of knowing that and was logging every reading as if it were brand new. Left as-is, the model would eventually have picked up on stretches of perfectly flat AQI that were never actually real, just the same stale number being read again and again.

After talking this through, the fix settled on was: switch the primary source for both live readings and historical backfill to Open-Meteo, a free API that updates hourly with real variation and reports every pollutant category, and keep the ground station around only as a secondary check. On top of that, a staleness check was added that compares each new ground-station reading against the last few stored values and flags it when they're identical. All three

cases were tested directly a stuck reading, a normal fresh one, and what happens if the primary source goes down and the system has to fall back.

This also explained a separate, smaller issue on the live dashboard: several pollutants were showing up as exactly 0.0 µg/m³, which looked like a bug but was actually missing data being displayed as if it were zero. The Karachi ground station only measures PM2.5, so anything else it was asked for came back empty, and that empty value was rendered as zero. Once the current-reading endpoint switched to Open-Meteo, which reports real numbers across all six categories, this cleared up on its own.

8. Testing and Verification

- Every backend change was compiled and, where practical, executed against representative data before being considered complete
- The full pipeline was re-run end to end after each fix to confirm no earlier behavior had regressed
- The frontend was type-checked and production-built after every change, with zero outstanding errors at each delivery point
- A dedicated end to end test script exercises every API endpoint against the running backend before the frontend is considered ready to connect
- Specific failure scenarios (a stuck sensor reading, a missing schema column, a mixed time zone dataset) were reproduced directly in a controlled test before and after each fix, to confirm the fix addressed the actual root cause rather than only the symptom

9. Key Learnings

- **Distinguish transient from deterministic failures:** Retrying a network blip is correct; retrying a schema mismatch only delays discovering the real problem. Treating every failure, the same way either wastes time or hides real defects.
- **A successful run is not the same as a correct run:** A pipeline that silently falls back to a degraded path and still reports success is more dangerous than one that fails loudly, because it hides data loss until something downstream (in this case, model training) exposes it much later.
- **Keep training and serving features consistent:** Several issues traced back to the historical backfill and the live pipeline computing slightly different feature sets from different sources; aligning both on the same primary data source removed an entire category of schema and data-quality problems at once.
- **Validate data sources, not just code.:** A perfectly correct pipeline can still learn the wrong thing if the underlying sensor itself provides low-resolution or incomplete data; the fix here was in the data source selection, not in the modelling code.
- **Separating frontend and backend by a strict API contract paid off:** Because the frontend only ever spoke to documented REST endpoints, it was possible to discard and rebuild the entire frontend without a single backend change, and to verify the new frontend purely against the documented contract.

10. Final System Capabilities

Component	Detail
Historical training window	730 days (approx. 17,500 hourly records) per full backfill
Forecast horizons	24 hours, 48 hours, and 72 hours ahead
Candidate models per horizon	Random Forest, Ridge Regression, and a recurrent neural network
Evaluation metrics	RMSE, MAE, and R^2 on a held-out test split
Automated schedule	Feature ingestion hourly; model retraining daily; backfill on demand

11. Dashboard Walkthrough

The dashboard is organized into four tabs, each backed by its own set of API calls. The table below describes what each tab shows, where its data comes from, and what to expect in terms of load time including which operations are noticeably slower than the rest of the interface.

Tab	What it shows	Data source	Typical load time
3-Day Forecast (home)	Live AQI and category, current weather, the three-day forecast as day cards plus a chart/table toggle, historical AQI trends (7/30/90-day filter), and inline hazard warnings	Live current-conditions endpoint, forecast endpoint, and history endpoint, called in parallel	Fast under normal conditions (well under a second per call); the 90-day historical view is the slowest part of this tab since it transfers the most data points
Feature Store	Total record count, feature-group metadata, a sample of recent feature rows, and a control to trigger a new historical backfill	Feature-store read endpoint	Fast to load and display; triggering a new backfill from this tab is a background job that itself takes several minutes to complete for a full historical window, though the tab remains responsive while it runs
Model Registry	Every candidate model's RMSE, MAE, and R^2 per forecast horizon, which candidate is currently active, the most recent training date, and controls to switch the active model or trigger retraining	Model-registry read endpoint	Fast to load; triggering retraining is the single slowest operation in the whole system, since it fits three model types across three horizons, and the neural-network candidate in particular takes noticeably longer to train than the other two

SHAP & Analytics	Real feature-importance values explaining the active model's prediction for each of the three forecast days	Explainability endpoint, computed on demand per day	Fast when the active model for a horizon is the Random Forest or Ridge Regression candidate; noticeably slower when the active model is the neural network, because explaining a neural network requires a slower, sampling-based approximation method rather than the near-instant method available for tree-based and linear models

Two points are worth calling out specifically. First, the two background job buttons, triggering a backfill from the Feature Store tab and triggering retraining from the Model Registry tab, both return an immediate acknowledgement while the actual work continues in the background; the dashboard does not block or show a long spinner for these, but the underlying job can take anywhere from under a minute (retraining, if the neural-network candidate trains quickly) to several minutes (a full historical backfill). Second, within the SHAP & Analytics tab, explanation speed depends entirely on which model type currently holds the "active" position for that horizon, since the neural-network candidate requires a fundamentally slower explanation technique than the two other candidate types.

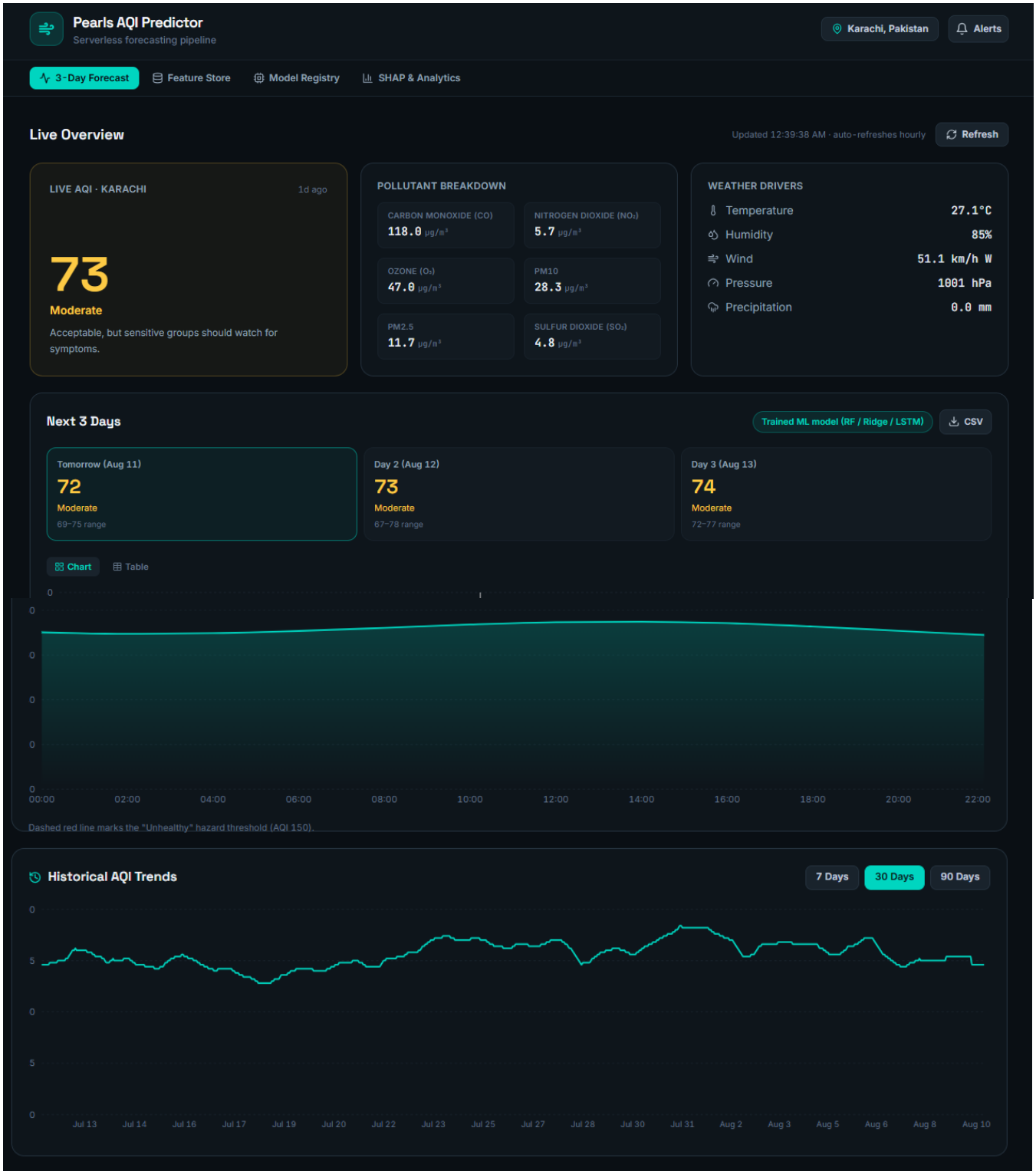
12. Limitations

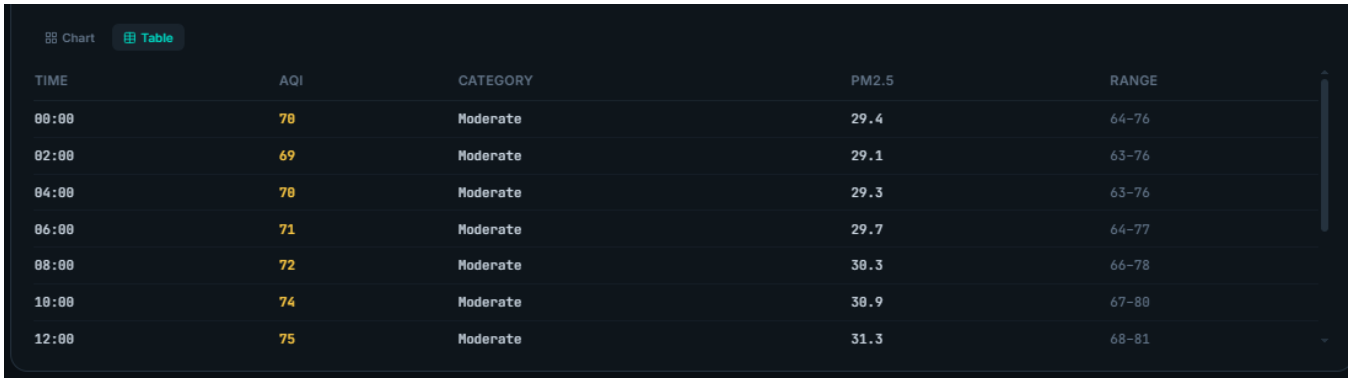
- **Single-city scope:** The system is built and verified for Karachi only. Extending it to another city currently means re-pointing the same feature group and models at new coordinates, which overwrites Karachi's trained models rather than running alongside them; genuine multi-city support would require a separate feature group and model set per city.
- **Local dashboard and scheduled training can drift out of sync:** The Model Registry view reads training results from a local file. When training runs on the scheduled automation rather than on the machine running the dashboard, its results reach Hopsworks immediately but only reach that local file when training is re-run locally or its output is fetched manually.
- **Ground-station data coverage in Karachi is limited:** The AQICN ground station used only reports one pollutant category and updates on a multi-hour cycle rather than hourly, which is why it was demoted to a secondary, cross-check role rather than the primary data source (see Section 7).
- **Historical depth is bounded by the data source:** The archive used for backfill only extends back to mid-2022, which caps how far into the past the training window can reach regardless of the requested number of days.
- **No automated data-drift monitoring beyond staleness detection:** The system checks for one specific failure pattern (a sensor repeating its last value) but does not currently monitor for broader distribution shift in incoming data over time.
- **Hyper-parameters were not extensively tuned:** Each candidate model uses a reasonable default configuration; a systematic search was out of scope for this phase and is a natural next step for improving forecast accuracy further.
- **Single-user design:** The dashboard has no authentication or per-user state; it is built as a single shared view of one city's forecast, not a multi-tenant product.
- **Development-environment sensitivity:** Several of the issues in Section 5 were specific to running parts of the pipeline on Windows; the system runs most reliably on Linux, which is what the scheduled automation uses.

13. Project Screenshots

This section is a placeholder for visual evidence of the working system drop in screenshots from an actual run to fill it out.

13.1 Home / 3-Day Forecast tab





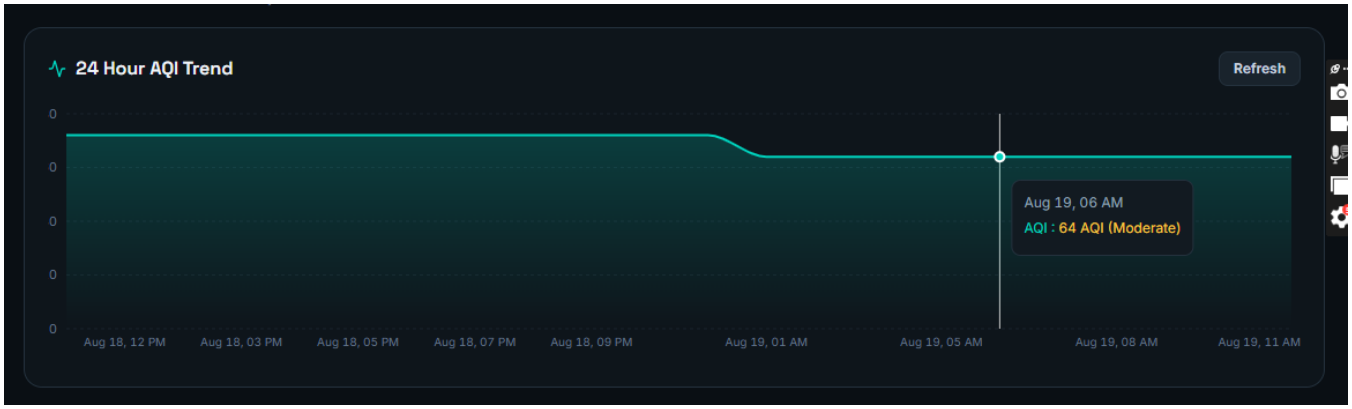


Fig 1: Shows the live AQI card, current weather, the three-day forecast cards with chart/table toggle, 24 Hours AQI trend and the historical trends chart.

13.2 Feature Store tab

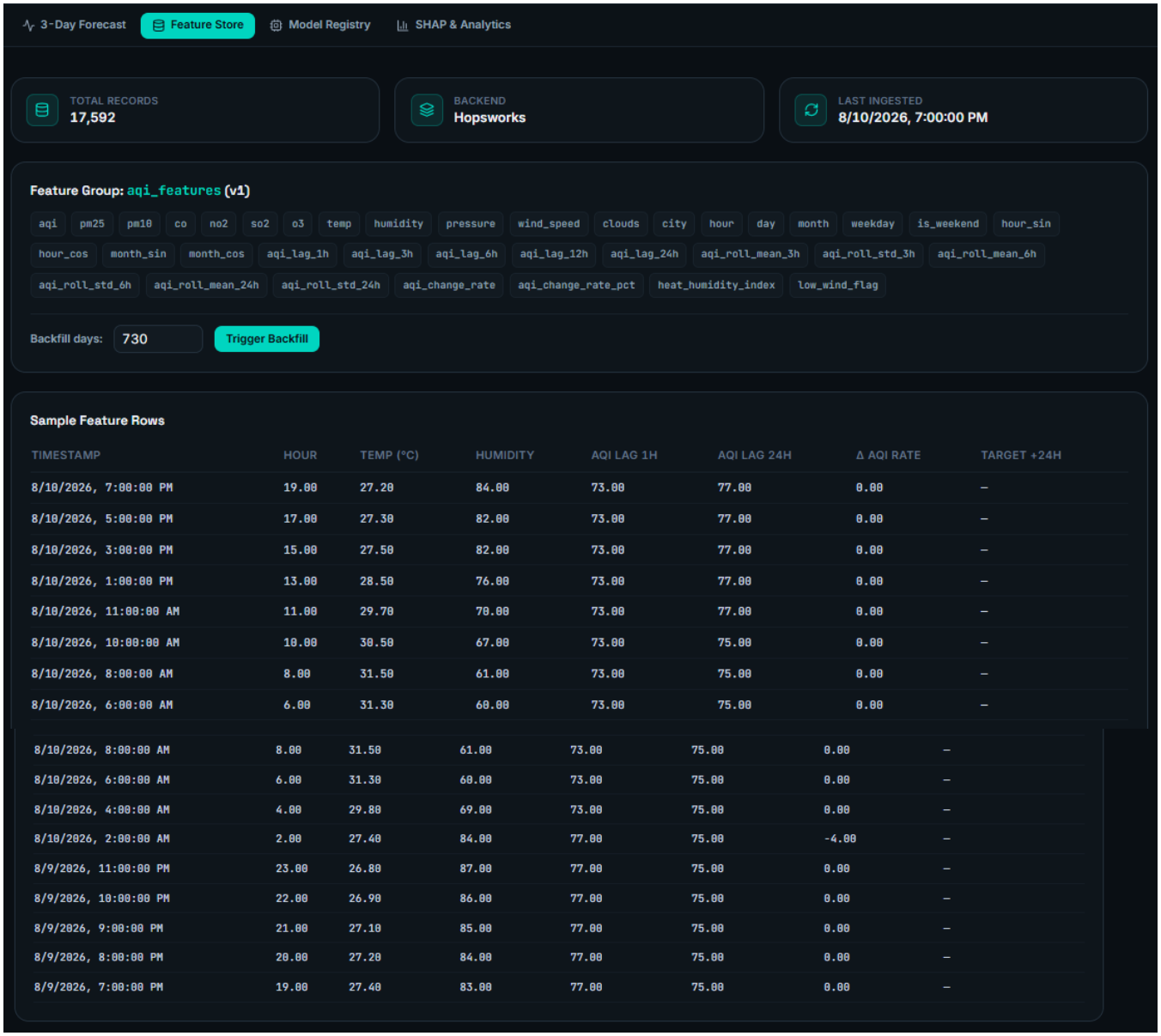


Fig 2: Shows total record count, feature-group metadata, and a sample of recent feature rows.

13.3 Model Registry tab

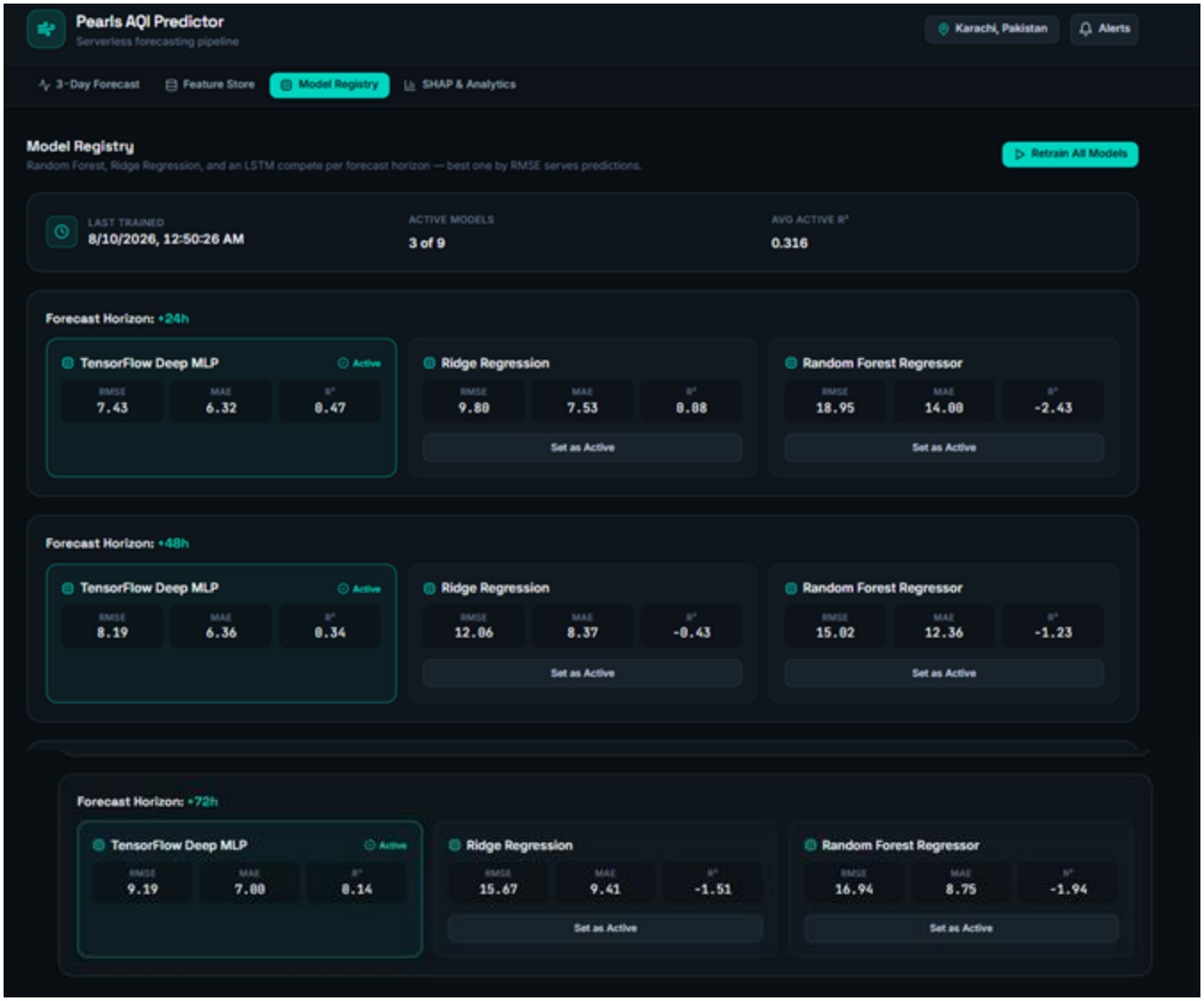
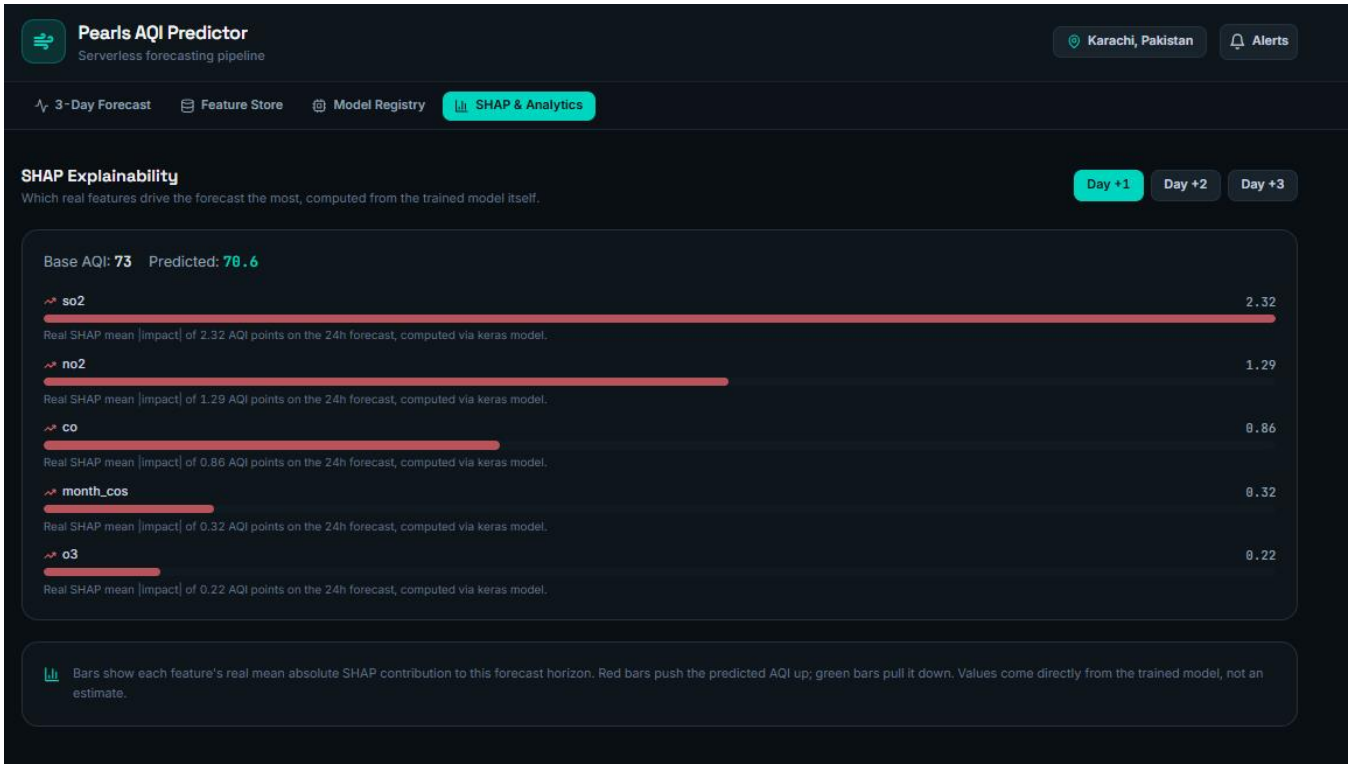
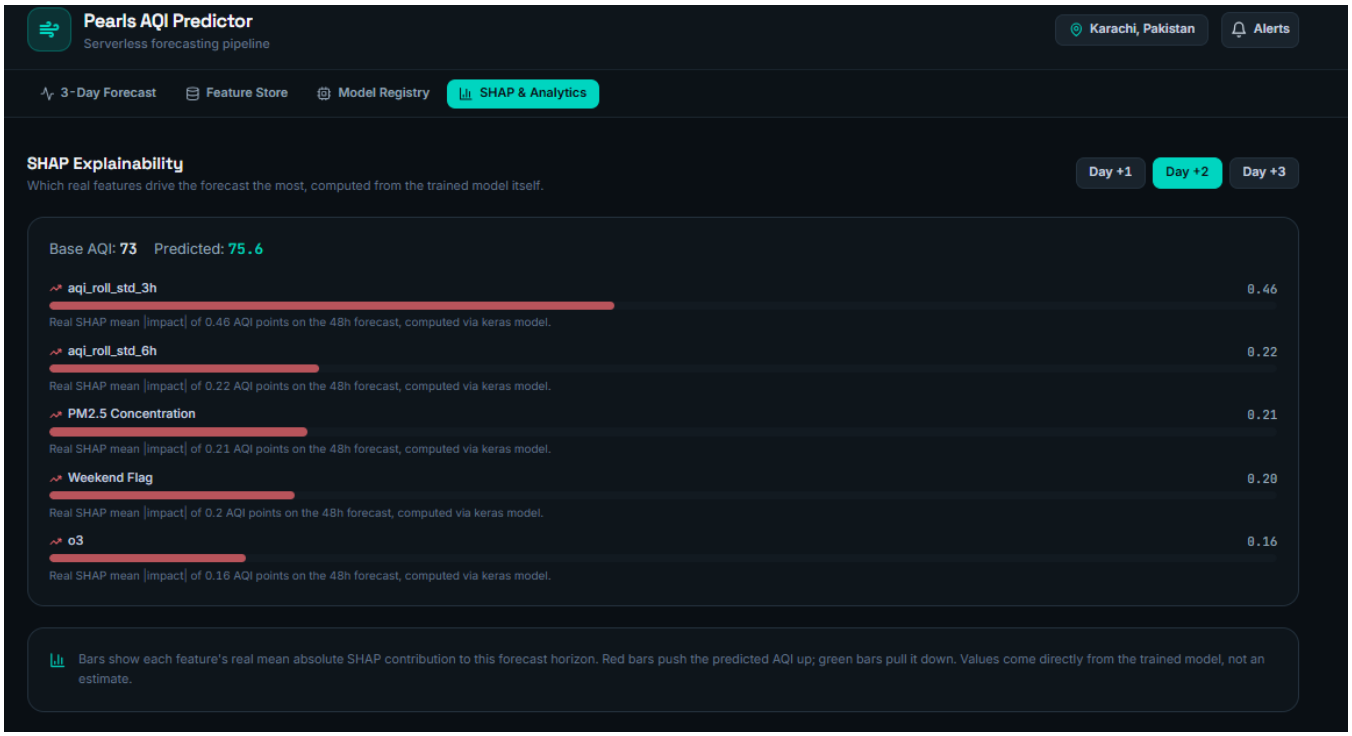


Fig 3: Shows RMSE/MAE/R^2 per candidate model and horizon, the active model, and the last-trained date.

13.4 SHAP & Analytics tab





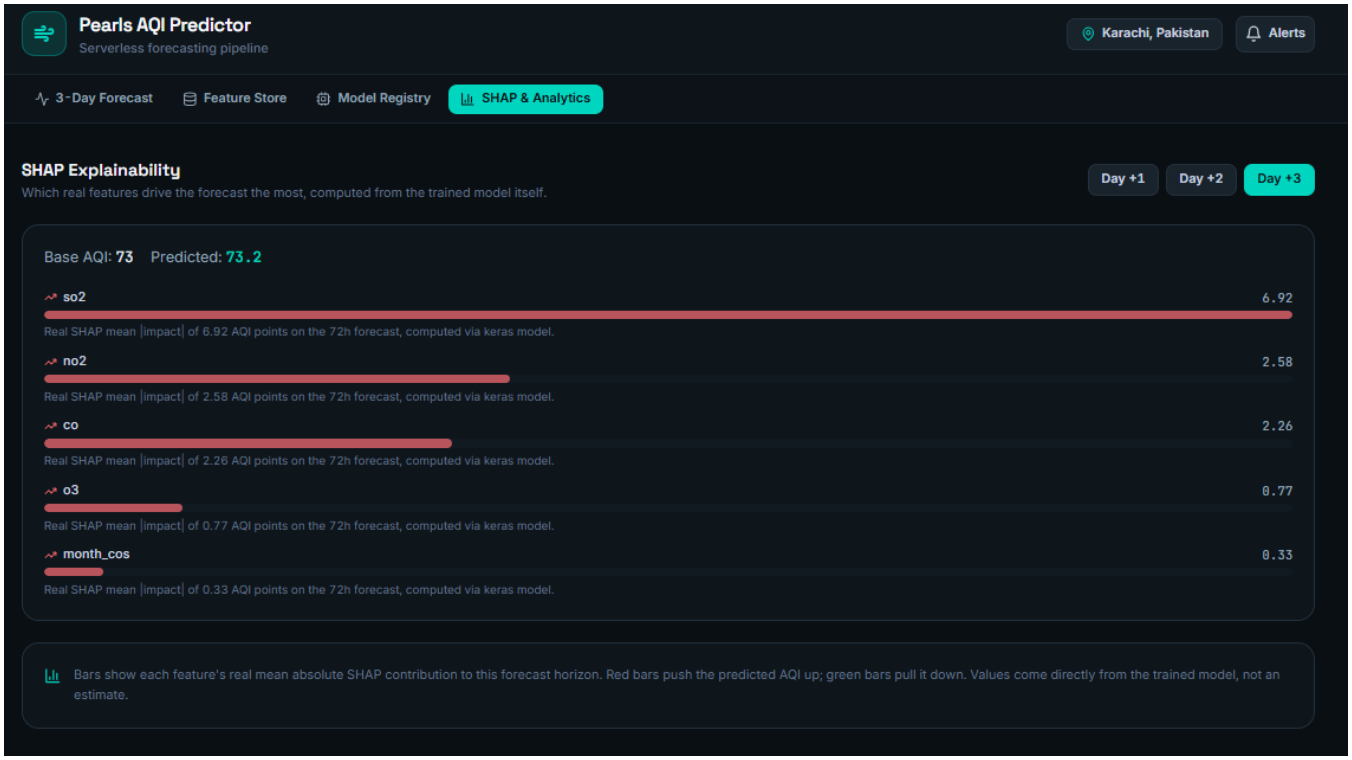


Fig 4: Shows real feature-importance values for the active model's forecast.

13.5 Hazard alert example

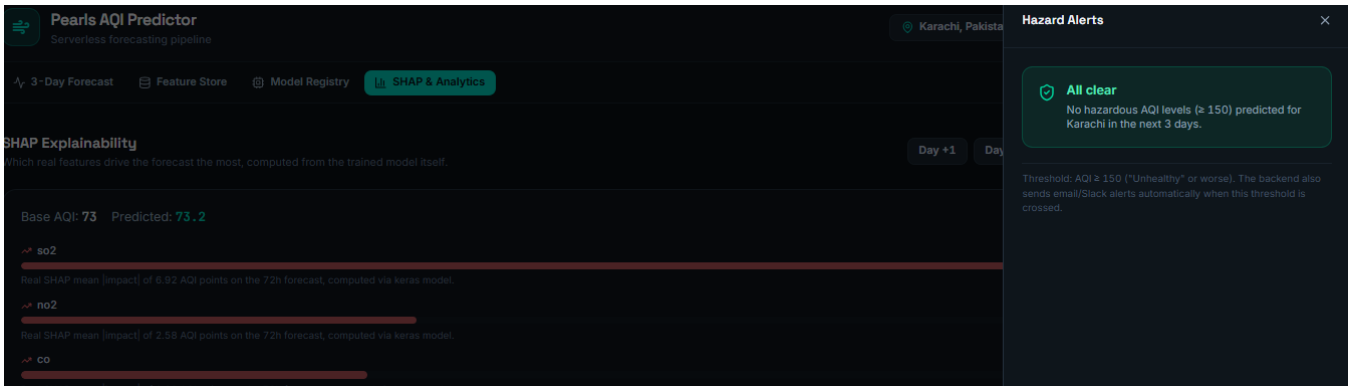


Fig 5: Shows the alert panel or inline banner when a hazardous AQI threshold is crossed.

14. Conclusion

What exists now is a full pipeline that starts with raw data from real external sources and ends with a live dashboard, running on automation that doesn't depend on any single machine being switched on. Getting there took a lot more debugging than expected, especially around Hopsworks, but the approach stayed consistent throughout: reproduce the problem, figure out whether it was a one-off blip or something that would keep happening, fix the actual cause, and check the fix actually held before moving to the next thing. The payoff is a system where every number on the

dashboard comes from a real computation, and where the failures that came up during development are now caught and handled automatically instead of needing someone to notice and step in.